Анализ данных на Python: конспект с нуля

Полный разбор курса: pandas, объединение таблиц, сводные таблицы, визуализация, описательные статистики, выбросы и пропуски, корреляция, линейная регрессия, тестирование гипотез. Код приводится в том виде, в каком он дан в лекциях.

0. Общая логика работы

Любой анализ идёт по одной цепочке. Полезно всё время понимать, на каком шаге вы находитесь:

Загрузка данных → осмотр → объединение таблиц → чистка (пропуски, выбросы, дубликаты) → новые признаки → агрегирование и сводные таблицы → визуализация → статистика (корреляция, гипотезы, регрессия) → выводы.

1. Структуры данных в pandas

DataFrame — таблица: есть индекс (номера строк) и один или несколько признаков (столбцов). **Series** — таблица из индекса и одного признака. Каждый отдельный столбец датафрейма — это Series.

```
import pandas as pd

df = pd.read_csv('путь/country.csv')   # загрузить таблицу
df.head()        # первые 5 строк
df.shape         # (количество строк, количество столбцов)
df.dtypes        # типы признаков
df.index         # индексы строк
```

Как обратиться к данным

Что нужно	Код
Один признак (Series)	`df['признак']`
Несколько признаков (DataFrame)	`df[['признак 1', 'признак 2']]`
Одно наблюдение (строка)	`df.loc[индекс]`
Одно значение	`df.loc[индекс]['признак']`

Типы данных

`int64` — целые числа, `float64` — дробные, `object` — текст/категории, `bool` — True/False, `datetime64` — дата и время.

Новый признак и фильтрация

```
# новый столбец считается сразу для всех строк (циклы не нужны)
df['total_price'] = df['transaction_qty'] * df['unit_price']

# фильтрация: условие даёт столбец True/False, df[...] оставляет строки с True
df[df['GDP'] > 100]
df[(df['GDP'] > 100) & (df['dem'] == 1)]    # & = И, | = ИЛИ, каждое условие в скобках
```

Шкалы признаков

Тип признака	Что это	Что корректно считать
Количественный	числа с осмысленной арифметикой (ВВП, возраст, цена)	среднее, медиана, дисперсия, std, квартили, размах
Категориальный	метки без порядка (регион, профессия, тип жилья)	мода, количество уникальных значений
Бинарный	ровно два значения (пол, да/нет, 0/1)	мода; среднее по 0/1 = доля единиц

Ловушка. ID (например `SK_ID_CURR`) записан числом, но это метка, а не величина: признак категориальный, среднее от него бессмысленно.

2. Даты

Сначала текст превращаем в дату, иначе Python видит просто строку символов. После этого через `.dt` достаём любые компоненты.

```
df['DateTime'] = pd.to_datetime(df['DateTime'])

df['year']  = df['DateTime'].dt.year
df['month'] = df['DateTime'].dt.month
df['day']   = df['DateTime'].dt.day
df['hour']  = df['DateTime'].dt.hour
df['dow']   = df['DateTime'].dt.dayofweek          # 0 = понедельник
df['week']  = df['DateTime'].dt.isocalendar().week # номер недели

df['DateTime'].min(), df['DateTime'].max()   # временной промежуток данных
```

Признак по условию строится функцией `np.where(условие, значение если да, значение если нет)` — это «если» сразу для всего столбца:

```
import numpy as np
df['period'] = np.where(df['DateTime'].dt.hour < 12, 'AM', 'PM')

# три категории — вложенный np.where
q1 = df['income'].quantile(0.25)
q3 = df['income'].quantile(0.75)
df['CODE_INCOME'] = np.where(df['income'] <= q1, 'low',
                     np.where(df['income'] <= q3, 'medium', 'high'))
```

3. Объединение таблиц

3.1 Вертикальное объединение: concat

Склейка однотипных таблиц друг под друга (Европа + Африка; продажи за первое полугодие + за второе).

```
pd.concat([A, B])
pd.concat([A, B], ignore_index=True)   # сквозная нумерация строк
```

Важно. Склейка идёт по названиям столбцов. Если в таблице B столбец называется `name`, а в A — `country`, pandas решит, что это разные признаки, и половина таблицы заполнится NaN. Сначала переименуйте:

```
B = B.rename(columns={'name': 'country'})
pd.concat([A, B])
```

3.2 Горизонтальное объединение: merge

Подтягивание столбцов из другой таблицы по общей метке — **якорю**.

```
pd.merge(левый датафрейм, правый датафрейм, якорь, how=...)

pd.merge(A, B, on='country', how='left')   # якорь — одноимённый признак в обеих
```

Якорь задаётся в зависимости от того, где лежит общая метка:

Ситуация	Код
Признак в обеих таблицах, назван одинаково	`on='признак'`
Признак в обеих, названия разные	`left_on='признак', right_on='признак'`
Слева индекс, справа признак	`left_index=True, right_on='признак'`
Слева признак, справа индекс	`left_on='признак', right_index=True`
Индекс в обеих	`left_index=True, right_index=True`

Параметр how — какие наблюдения останутся:

how	Что остаётся	Когда использовать
`'inner'` (по умолчанию)	только наблюдения, которые есть в обеих таблицах	«найдите то, что есть одновременно в A и B»
`'left'`	все наблюдения левой таблицы; чего нет справа — NaN	«для каждого объекта таблицы A найдите данные из B»
`'right'`	все наблюдения правой таблицы	зеркально предыдущему
`'outer'`	все наблюдения обеих таблиц	нужно ничего не потерять

3.3 Дубликаты перед объединением

```
df.duplicated().sum()        # количество полностью повторяющихся строк
df = df.drop_duplicates()    # удалить
```

Зачем. Если в правой таблице ключ повторяется дважды, каждая подходящая строка левой таблицы после merge тоже удвоится: структура данных ломается, суммы задваиваются. Поэтому справочники чистят *до* объединения.

Проверка качества объединения: `len(df)` до и после (не должно вырасти) и `df.isna().sum()` (NaN = ключи, для которых пары не нашлось).

4. Агрегирование и сводные таблицы

Агрегирование — сведение множества значений к одному числу.

Метод	Что считает
`.min()`, `.max()`	минимум, максимум
`.sum()`	сумму
`.mean()`, `.median()`	среднее, медиану
`.count()`	количество непропущенных значений (строк)
`.nunique()`	количество **уникальных** значений
`.mode()[0]`	самое частое значение
`.shape[0]`	количество строк всего, включая NaN

count против nunique. Один заказ состоит из нескольких товаров и занимает несколько строк. Поэтому «количество заказов» — это всегда `nunique` по transaction_id, а `count` дал бы количество позиций в чеках. Это самая частая ошибка в кейсах.

4.1 Сводная таблица — что это

Таблица, агрегирующая данные по категориям одного или нескольких признаков. Неумный способ — считать каждую категорию отдельным фильтром; умный — `groupby` или `pivot_table`. Эти два способа взаимозаменяемы.

```
# ПРОСТЕЙШАЯ ГРУППИРОВКА: средний ВВП для каждой системы правления
A.groupby('gov')['GDP'].mean()

A.pivot_table(index='gov',          # признак группировки (по строкам)
              values='GDP',          # признак агрегирования
              aggfunc='mean')        # метод агрегирования
```

```
# НЕСКОЛЬКО ПРИЗНАКОВ АГРЕГИРОВАНИЯ
A.groupby('gov')[['GDP', 'pop']].mean()
A.pivot_table(index='gov', values=['GDP', 'pop'], aggfunc='mean')
```

```
# НЕСКОЛЬКО ПРИЗНАКОВ ГРУППИРОВКИ: внутри каждого региона — по каждой системе правления
A.groupby(['reg', 'gov'])['GDP'].mean()
A.pivot_table(index=['reg', 'gov'], values='GDP', aggfunc='mean')
```

```
# НЕСКОЛЬКО МЕТОДОВ АГРЕГИРОВАНИЯ для одного признака
A.groupby('gov')['GDP'].agg(['mean', 'max'])
A.pivot_table(index='gov', values='GDP', aggfunc=['mean', 'max'])
```

```
# РАЗНЫЕ МЕТОДЫ ДЛЯ РАЗНЫХ ПРИЗНАКОВ (словарь признак: метод)
A.groupby('gov').agg({'GDP': 'mean', 'pop': 'max'})
A.pivot_table(index='gov', aggfunc={'GDP': 'mean', 'pop': 'max'})
```

```
# СВОЯ ФУНКЦИЯ: например размах внутри группы
def coded(x):
    return x.max() - x.min()

A.groupby('gov')['GDP'].agg(coded)
A.pivot_table(index='gov', values='GDP', aggfunc=coded)
```

4.2 Разрез по строкам и столбцам (columns)

Второй признак можно вынести не в строки, а в столбцы — получится прямоугольная таблица, как в Excel. Это нужно, когда дальше строится линейный график: каждый столбец станет отдельной линией.

```
# по строкам дни, по столбцам месяцы, в ячейках — количество уникальных заказов
pivot = df.pivot_table(index='day',
                       columns='month',
                       values='transaction_id',
                       aggfunc='nunique')
pivot.plot()      # шесть линий, по одной на месяц
```

4.3 Деление одной сводной таблицы на другую

Если у двух сводных таблиц одинаковые строки и столбцы, их можно делить поэлементно. Так считается средний чек (AOV = выручка / количество уникальных транзакций):

```
revenue = df.pivot_table(index='month', columns='store_location',
                         values='total_price', aggfunc='sum')
orders  = df.pivot_table(index='month', columns='store_location',
                         values='transaction_id', aggfunc='nunique')
aov = revenue / orders
aov.plot()
```

4.4 Что делают с результатом

```
gb = df.groupby('store_location')['total_price'].sum()

gb.sort_values(ascending=False)   # отсортировать (нужно почти всегда перед графиком)
gb.idxmax()                       # НАЗВАНИЕ группы с максимумом
gb.max()                          # само максимальное значение
gb.head(3)                        # первые три
gb.reset_index()                  # превратить обратно в обычную таблицу со столбцами
```

Доля каждой категории внутри своей группы (например доля товара в выручке кофейни) считается через `transform`: он возвращает столбец той же длины, что исходная таблица, подставляя каждой строке итог её группы.

```
gb = df.groupby(['store_location', 'product_name'])['total_price'].sum().reset_index()
gb['share'] = gb['total_price'] / gb.groupby('store_location')['total_price'].transform('sum') * 100
```

5. Визуализация

Задача	График	Код
Распределение количественного признака	гистограмма	`df['X'].plot(kind='hist', bins=20)`
Распределение и выбросы	ящик с усами	`df['X'].plot(kind='box', vert=False)`
Динамика во времени	линейный	`df.plot(y='X')` / `pivot.plot()`
Связь двух количественных	диаграмма рассеяния	`df.plot('X', 'Y', kind='scatter')`
Сравнение категорий	столбчатая	`gb.plot(y='X', kind='bar')` или `kind='barh'`
Доли в составе целого	круговая	`gb.plot(y='count', kind='pie')`
Распределение по группам	boxplot / violinplot	`sns.boxplot(data=df, x='X', y='группа')`

Для линейного графика данные должны быть отсортированы по времени: `df.sort_values('год').plot(y='X')` или `df.sort_index().plot(y='X')`, иначе линия будет скакать.

Как читать графики в тестах

- **Гистограмма:** где самый высокий столбик и в какую сторону тянется хвост. Длинный правый хвост = правосторонняя скошенность (типично для денег).
- **Ящик с усами:** прямоугольник — от Q1 до Q3 (это IQR), черта внутри — медиана, усы — до границ нормы, точки за усами — выбросы. Точки только справа = скошенность вправо.
- **Скрипичный график:** ширина показывает плотность; симметричная скрипка — симметричное распределение, узкая и короткая — малый разброс.
- **Диаграмма рассеяния:** вытянутое облако вдоль прямой = сильная линейная связь; бесформенное облако = связи нет.

6. Описательные статистики

```
df.describe()                  # для int64 и float64
df.describe(include=object)    # для категориальных: count, nunique, top (мода), freq
```

Меры центральной тенденции (описывают типичное наблюдение)

Мера	Код	Особенности
Среднее арифметическое	`.mean()`	количественные; для бинарных 0/1 показывает долю единиц; **не устойчиво к выбросам**
Медиана	`.median()`	количественные; делит данные пополам; **устойчива к выбросам**
Мода	`.mode()[0]`	категориальные: самое частое значение

Правило: **среднее стремится к выбросам, медиана стремится от выбросов.** Поэтому для времени ожидания, зарплат, цен типичное значение описывает медиана.

Квартили

Квартиль	Код	Смысл
Q0	`.quantile(0)` = `.min()`	минимум
Q1	`.quantile(0.25)`	25% данных меньше, 75% больше
Q2	`.quantile(0.5)` = `.median()`	медиана, делит пополам
Q3	`.quantile(0.75)`	75% данных меньше, 25% больше
Q4	`.quantile(1)` = `.max()`	максимум

Меры вариативности (описывают разброс)

Мера	Код	Смысл и особенности
Количество уникальных значений	`.nunique()`	разброс категориального признака
Размах	`.max() - .min()`	диапазон 100% данных; не устойчив к выбросам
Интерквартильный размах (IQR)	`.quantile(0.75) - .quantile(0.25)`	диапазон 50% центральных данных; **наиболее устойчив к выбросам**
Дисперсия	`.var()`	усреднённый квадрат отклонения от среднего; не устойчива
Стандартное отклонение	`.std()`	усреднённое отклонение от среднего; не устойчиво

Словарь формулировок (как читать вопросы теста)

Формулировка в задании	Что считать
типичное наблюдение / типичная категория	медиана или мода
мера центральной тенденции, устойчивая к выбросам	медиана
амплитуда разброса	размах: max − min
диапазон, в котором лежит 50% центральных данных	IQR
на сколько в среднем отклоняется от среднего арифметического	стандартное отклонение `.std()`
усреднённый квадрат отклонения от среднего	дисперсия `.var()`
больше какого значения получает 50% объектов	медиана
какая доля объектов имеет признак (0/1)	`.mean()` по бинарному столбцу
мера вариативности категориального признака	`.nunique()`

7. Выбросы

Выбросы — экстремальные значения, сильно отличающиеся от основной массы. Видны на гистограмме, ящике с усами, скрипичном графике и диаграмме рассеяния.

Метод N·STD (для симметричных, колоколообразных данных)

```
col = df['признак']
avg = col.mean()
std = col.std()
left  = avg - 2*std
right = avg + 2*std

clean = df[col.between(left, right)]
```

Метод 1.5·IQR (универсальный, для скошенных данных)

```
col = df['признак']
q1  = col.quantile(0.25)
q3  = col.quantile(0.75)
iqr = q3 - q1
left  = q1 - 1.5*iqr
right = q3 + 1.5*iqr

clean = df[col.between(left, right)]
```

`col.between(left, right)` — это то же самое, что `(col >= left) & (col <= right)`, просто короче.

Логарифмирование

Деньги почти всегда скошены вправо: много мелких значений и длинный хвост крупных. Логарифм сжимает большие значения сильнее маленьких, и распределение становится похожим на колокол. Это нужно и для корректного отсечения выбросов методом STD, и для регрессии.

```
import numpy as np
df['ln_annuity'] = np.log(df['AMT_ANNUITY'])   # вперёд
np.exp(значение)                                # обратно

df['AMT_ANNUITY'].skew()    # асимметрия: было, например, 1.8
df['ln_annuity'].skew()     # стало -0.3, то есть почти симметрично
```

8. Пропуски

```
df.isna().sum()     # количество пропусков в каждом признаке
df.isna().mean()    # доля пропусков (умножить на 100 — проценты)
```

Работает это так: `isna()` даёт таблицу из True/False, а True считается за 1. Поэтому сумма = количество пропусков, а среднее = доля.

Чем заполнять

```
# количественный признак: среднее или медиана
df['AGE_CLEAN'] = df['AGE'].fillna(df['AGE'].mean())

# категориальный признак: мода
df['GENDER_CLEAN'] = df['CODE_GENDER'].fillna(df['CODE_GENDER'].mode()[0])
```

Более точный способ — заполнять статистикой по группе похожих наблюдений. Логика: если размер кредита сильно зависит от уровня дохода, то пустую ячейку логичнее заполнить медианой своей доходной группы, а не общей.

```
df['CREDIT_CLEAN'] = df['AMT_CREDIT'].fillna(
    df.groupby('CODE_INCOME')['AMT_CREDIT'].transform('median'))
```

Разница двух родственных методов: `.agg('median')` возвращает короткую таблицу-итог (по строке на группу), а `.transform('median')` — длинный столбец под размер исходной таблицы, где каждой строке подставлена медиана её группы. Именно поэтому transform можно сразу класть в `fillna`.

Для моды по группам:

```
modes = df.groupby('CODE_INCOME')['NAME_INCOME_TYPE'].agg(lambda s: s.mode()[0])
df['TYPE_CLEAN'] = df['NAME_INCOME_TYPE'].fillna(df['CODE_INCOME'].map(modes))
```

9. Корреляция

Корреляция — мера статистической связи между двумя признаками. Значения от −1 до 1.

- Положительная: если A растёт, то и B растёт; если A падает, то и B падает.
- Отрицательная: если A растёт, B падает, и наоборот.
- Около нуля (примерно от −0.1 до 0.1): связи нет.

> **Главное правило интерпретации.** Корреляция не говорит о причинно-следственной связи. Слова «влияет», «зависит», «причина», «приводит к» в выводе некорректны. Корректно: «существует связь», «связаны», «чем больше A, тем больше B».

Пирсон и Спирман

	Пирсон	Спирман
Что измеряет	линейную связь	монотонную связь (растёт/убывает, не обязательно по прямой)
Для каких признаков	только количественные	количественные и категориальные порядковые
Выбросы	чувствителен	устойчив
Код	`df['A'].corr(df['B'])`	`df['A'].corr(df['B'], method='spearman')`

Поэтому на графике с явной, но изогнутой монотонной кривой (степенная, корень) Спирман будет заметно больше Пирсона по модулю. На немонотонных зависимостях (парабола, синусоида, круг) оба коэффициента близки к нулю, хотя связь в данных есть.

Корреляционная матрица

```
df.corr(numeric_only=True)                       # Пирсон
df.corr(numeric_only=True, method='spearman')    # Спирман

cm = df.corr(method='spearman')
cm['CO2'].sort_values(key=abs)   # какой признак сильнее связан с CO2 (без учёта знака)
```

По главной диагонали стоят единицы (корреляция признака с самим собой), матрица симметрична относительно диагонали.

Как формулировать вывод

Шаблон: *сила* + *направление* + *тип связи*. Пирсон 0.74 → «сильная положительная линейная связь». Спирман −0.75 → «сильная отрицательная монотонная связь». Пирсон 0.002 → «линейная связь практически отсутствует» (но не «связи нет вообще»: нелинейная связь может быть).

10. Линейная регрессия

Метод моделирования зависимости количественной целевой переменной Y от одного или нескольких предикторов X.

$\mathbf{Y = w_0 + w_1 \cdot X_1 + \ldots + w_n \cdot X_n}$, где w_0 — свободный коэффициент, w_i — коэффициенты предикторов.

Подбор предикторов

- Берём X с самой сильной связью с Y (смотрим корреляционную матрицу).
- Между самими предикторами сильной корреляции избегаем: это мультиколлинеарность, предикторы дублируют друг друга.
- Минимум 100 наблюдений на каждый предиктор.
- Предпочтительны признаки с симметричным распределением; несимметричные по возможности логарифмируем.

Деление выборки и обучение

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X = df[['предиктор 1', 'предиктор 2']]
Y = df['целевая']

X_train, X_test, y_train, y_test = train_test_split(X, Y,
                                                    test_size=0.2,
                                                    random_state=42)

model = LinearRegression().fit(X_train, y_train)
model.intercept_   # w0
model.coef_        # массив w1 ... wn в порядке столбцов X
```

Тренировочная выборка нужна для обучения (поиска коэффициентов методом наименьших квадратов), тестовая — для честной оценки качества. `random_state` фиксирует случайное разбиение, чтобы результат воспроизводился.

Метрики качества (считаются на тестовой выборке)

```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

y_pred = model.predict(X_test)
r2_score(y_test, y_pred)
mean_squared_error(y_test, y_pred)
mean_absolute_error(y_test, y_pred)
```

Метрика	Смысл	Значения
R^2 (коэффициент детерминации)	доля разброса целевой переменной, которую объясняет модель	от 0 до 1; чем ближе к 1, тем лучше. $R^2 = 0.85$ означает «объясняет 85% дисперсии»
MSE	усреднённый квадрат ошибки	от 0 до ∞; меньше — лучше; для сравнения моделей
MAE	усреднённая абсолютная ошибка, в единицах Y	от 0 до ∞; меньше — лучше. Именно MAE читается как «в среднем модель ошибается на …»

MSE и MAE можно сравнивать только между моделями с одинаковой целевой переменной. Если в одной модели Y в долларах, а в другой ln(Y), их ошибки несравнимы. R^2 сравнивать можно.

Интерпретация коэффициентов

Зависит от того, что логарифмировано. Запомните четыре случая:

Форма модели	Как читается коэффициент w_1
$Y = w_0 + w_1 \cdot X$	X вырос на 1 → Y меняется на w_1 единиц
$Y = w_0 + w_1 \cdot \ln(X)$	X вырос на 1% → Y меняется на $w_1/100$ единиц
$\ln(Y) = w_0 + w_1 \cdot X$	X вырос на 1 → Y меняется на $100 \cdot w_1$ %
$\ln(Y) = w_0 + w_1 \cdot \ln(X)$	X вырос на 1% → Y меняется на w_1 % (эластичность)

Для бинарного предиктора (0/1) формулировка меняется с «вырос на 1» на «равен 1»: например в модели ln(Y) при $w = -0.4941$ вывод «если X равен 1, это снижает Y на 49.41%».

Свободный коэффициент: если все предикторы равны нулю, Y равен w_0 (а в лог-модели — e^{w0}). Часто смысла не имеет.

Знак читается прямо: положительный коэффициент — рост, отрицательный — снижение. Коэффициент, равный нулю, означает, что предиктор ничего не даёт, и модель эквивалентна регрессии на константу.

Предсказание

```
model.predict([[значение X1, значение X2, ...]])

# если целевая была логарифмирована, результат нужно вернуть обратно:
np.exp(model.predict([[...]]))
```

11. Тестирование гипотез

Основные понятия

- **Генеральная совокупность** — все объекты, интересные в рамках исследования.
- **Выборка** — объекты, которые реально попали в исследование.
- **Репрезентативность** — свойство выборки отражать генеральную совокупность.
- **Статистическая гипотеза** — предположение о свойствах генеральной совокупности, которое проверяют по выборке.

Четыре шага

1. Формулируем H_0 и H_1.
2. Подбираем тест (по предпосылкам).
3. Проводим тест, получаем p-value.
4. Делаем вывод.

H_0 (нулевая) всегда формулируется через равенство: различий нет, среднее равно такому-то значению. **H_1** (альтернативная) бывает двусторонней (≠, `alternative='two-sided'`), правосторонней (>, `'greater'`/`'larger'`) и левосторонней (<, `'less'`/`'smaller'`).

Вывод по p-value

p-value — вероятность получить наблюдаемый результат, если H_0 верна. **Уровень значимости α** — вероятность отвергнуть H_0, когда она верна; стандартно 1%, 5%, 10%.

Условие	Вывод
p-value < α	H_0 отвергается (различия статистически значимы)
p-value > α	H_0 не отвергается

Пример: p-value = 0.02. На уровне 10% и 5% H_0 отвергается, на уровне 1% — не отвергается. Поэтому в тестах всегда проверяйте все три уровня.

Ошибки I и II рода

	H_0 верна	**H_0 неверна**
Приняли H_0	верно	ошибка II рода (вероятность β)
Отвергли H_0	ошибка I рода (вероятность α)	верно

Ошибка I рода — «увидели различие, которого нет». Ошибка II рода — «не заметили реальное различие».

Нормальное распределение и проверка на нормальность

Нормальное распределение $N(\mu, \sigma^2)$ симметрично, с пиком посередине; задаётся математическим ожиданием μ и дисперсией σ^2. Нормальность — предпосылка большинства тестов, её проверяют тестом Шапиро-Уилка:

```
from scipy.stats import shapiro
res = shapiro(X)
res[1]    # p-value
```

H_0: признак распределён нормально. Если p-value больше уровня значимости, H_0 не отвергается, и можно считать распределение нормальным.

Какой тест выбрать

Задача	**Предпосылки**	**Тест и код**
Сравнить среднее выборки с числом, дисперсия генеральной совокупности известна, n ≥ 30	признак нормален	Z-тест: `ztest(df['x'], value=34, alternative='two-sided')` из `statsmodels.stats.weightstats`
Сравнить среднее выборки с числом	признак нормален	Одновыборочный t-тест: `stats.ttest_1samp(df['x'], popmean=34, alternative='greater')`
Сравнить «до» и «после» на одних и тех же объектах	выборки связаны, их разность нормальна	Парный t-тест: `stats.ttest_rel(df['before'], df['after'], alternative='less')`
Сравнить две независимые группы	выборки не связаны, обе нормальны, дисперсии равны	Двухвыборочный t-тест: `stats.ttest_ind(df['with'], df['without'], alternative='two-sided')`
Сравнить две независимые группы, когда нормальности нет	выборки не связаны, форма распределений схожа	Тест Манна-Уитни: `stats.mannwhitneyu(df['with'], df['without'], alternative='two-sided')`
Сравнить доли в двух группах	выборки не связаны, признак бинарный	Z-тест для пропорций: `proportions_ztest(gb['sum'], gb['count'])`
Проверить связь двух категориальных признаков	в каждой группе не менее 5 наблюдений	χ^2-критерий: `stats.chi2_contingency(gb.values)`

Подготовка данных для последних двух тестов:

```
# для пропорций: сколько положительных исходов и сколько всего в каждой группе
gb = df.groupby('subsidies')['goal'].agg(['sum', 'count'])
from statsmodels.stats.proportion import proportions_ztest
proportions_ztest(gb['sum'], gb['count'], alternative='two-sided')

# для хи-квадрат: таблица сопряжённости
gb = df[['regime', 'agreement']].value_counts().unstack()
from scipy import stats
stats.chi2_contingency(gb.values)
```

Все тесты возвращают пару (статистика, p-value); решение принимается только по p-value.

12. Итоговая шпаргалка кода

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns

df = pd.read_csv('file.csv')
df.shape; df.head(); df.dtypes; df.describe()

# пропуски
df.isna().sum(); df.isna().mean()
df['x'] = df['x'].fillna(df['x'].median())

# дубликаты
df.duplicated().sum(); df = df.drop_duplicates()

# объединение
pd.concat([A, B], ignore_index=True)
pd.merge(A, B, on='key', how='left')

# даты
df['d'] = pd.to_datetime(df['d']); df['month'] = df['d'].dt.month

# новый признак
df['new'] = np.where(df['x'] < 10, 'low', 'high')

# агрегирование
df.groupby('g')['x'].mean()
df.pivot_table(index='g', columns='h', values='x', aggfunc='nunique')

# выбросы
col = df['x']; q1, q3 = col.quantile(0.25), col.quantile(0.75); iqr = q3 - q1
clean = df[col.between(q1 - 1.5*iqr, q3 + 1.5*iqr)]

# графики
df['x'].plot(kind='hist', bins=20)
df['x'].plot(kind='box', vert=False)
gb.sort_values().plot(kind='barh')
pivot.plot()

# корреляция
df['a'].corr(df['b']); df.corr(numeric_only=True, method='spearman')

# регрессия
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
model = LinearRegression().fit(X_train, y_train)
r2_score(y_test, model.predict(X_test))

# гипотезы
stats.ttest_ind(a, b, alternative='two-sided')
```